

Golang Profile Guided Optimization

HUANG Yanshuo
DevOps

Table of Content

- What is Profile Guided Optimization (PGO) ?
- How will PGO help?
 - Function Inlining
 - Devirtualization of interface call
- Benefits of PGO In Real World
- How to apply PGO ?
- Introducing AutoFDO

What is PGO ?

Profile Guided Optimization

- Also known as Feedback-Directed Optimization (FDO)
- Analyze the online profile
- Help compiler make optimization decisions
- Performance increases 2-14%
- Easy to implement

```
Showing nodes accounting for 177.69s, 99.22% of 179.08s total
Dropped 35 nodes (cum <= 0.90s)
Showing top 10 nodes out of 22
```

flat	flat%	sum%	cum	cum%	
116.65s	65.14%	65.14%	135.72s	75.79%	main.processWorkLoad (i
21.35s	11.92%	77.06%	157.09s	87.72%	main.main
19.01s	10.62%	87.68%	19.03s	10.63%	main.SlowProcessor.Proc
18.70s	10.44%	98.12%	18.73s	10.46%	runtime.pthread_cond_si
1.78s	0.99%	99.11%	1.78s	0.99%	runtime.pthread_cond_wa
0.09s	0.05%	99.16%	19.13s	10.68%	runtime.startm
0.05s	0.028%	99.19%	18.91s	10.56%	runtime.semawakeup
0.05s	0.028%	99.22%	19.64s	10.97%	runtime.wakeup
0.01s	0.0056%	99.22%	18.92s	10.57%	runtime.notewakeup
0	0%	99.22%	2.32s	1.30%	runtime.findRunnable

```
(pprof) list main.processWorkLoad
```

```
Total: 179.08s
```

```
ROUTINE ===== main.processWorkLoad in /Users/hys/De
```

```
116.65s 135.72s (flat, cum) 75.79% of Total
```

```
56:func processWorkLoad(w WorkLoad) int {
57:    sum := 0
58:    // Hot loop with interface call
59:    for _, d := range w.data {
60:        sum += w.processor.Process(d)
61:    }
62:    return sum
63:}
64:
65:func main() {
```


How PGO works ?

- **Function Inlining**
- **Devirtualization of Interface Call**

How PGO works ?

Function Inlining

- What is function inlining ?

```

✓ func add(x, y int) int { 3 usages new *
  — return x + y
}

✓ func sum(n int) int { 1 usage new *
  — v := 0
  — for i := 0; i < n; i++ {
    — | — v = add(v, i)
    — }
  — return v
}

```

Becomes



```

✓ func sum(n int) int { 1 usage new *
  — v := 0
  — for i := 0; i < n; i++ {
    — | — v = v + i
    — }
  — return v
}

```

Benefits of Function Inlining

Benchmark Result

Add-11	0.2694n ± 8%
AddNoInline-11	0.8388n ± 5%
SumWithInline-11	275.5n ± 4%
SumWithoutInline-11	829.0n ± 2%

x3 Faster

Benefits of Function Inline

Assembly Deep Dive with -gcflags -S

w/o inline

```

— FUNCDATA— $0, gclocals·g2BeySu+wFnoycgXfElmcg==(SB)
— FUNCDATA— $1, gclocals·g2BeySu+wFnoycgXfElmcg==(SB)
— FUNCDATA— $5, main.sumWithoutInline.arginfo1(SB)
— FUNCDATA— $6, main.sumWithoutInline.argliveinfo(SB)
— PCDATA— $3, $1
— MOVD— R0, main.n(FP)
— PCDATA— $3, $-1
— MOVD— ZR, R1
— MOVD— ZR, R2
— JMP- 68
— MOVD— R1, main.i-8(SP)
— MOVD— R2, R0
— PCDATA— $1, $0
— CALL— main.addNoInline(SB)
— MOVD— main.i-8(SP), R2
— ADD- $1, R2, R1
— MOVD— R0, R2
— MOVD— main.n(FP), R0
— CMP- R1, R0
— BGT- 40

```

w/ inline

```

— FUNCDATA— $0, gclocals·g2BeySu+wFnoycgXfElmcg==(SB)
— FUNCDATA— $1, gclocals·g2BeySu+wFnoycgXfElmcg==(SB)
— FUNCDATA— $5, main.sumWithInline.arginfo1(SB)
— FUNCDATA— $6, main.sumWithInline.argliveinfo(SB)
— PCDATA— $3, $1
— MOVD— ZR, R1
— MOVD— ZR, R2
— JMP- 28
— ADD- $1, R1, R3
— HINT— $0
— ADD- R1, R2, R2
— MOVD— R3, R1
— CMP- R1, R0
— BGT- 12
— MOVD— R2, R0
— RET- (R30)

```

How PGO works ?

Inline Hot Function Calls

- Example from official <https://go.dev/blog/pgo>
- Benchmark on a Markdown HTTP Server
- Steps:
 1. `go build -o markdown.nopgo`
 2. Run load generator and get the cpu profile
 3. `go build -pgo cpu.pprof -o markdown.withpgo`
 4. Run again, and get the cpu profile
 5. Compare

How PGO works ?

Inline Hot Function Calls

- CPU optimization:
 - -9.33s, 8.42%

```
$ go tool pprof -diff_base cpu.nopgo.pprof cpu.pgo.pprof
File: markdown.pgo
Type: cpu
Time: Feb 16, 2025 at 8:50pm (CST)
Duration: 60.21s, Total samples = 110.75s (183.93%)
Entering interactive mode (type "help" for commands, "o" for options)
(pprof) top
Showing nodes accounting for -9.33s, 8.42% of 110.75s total
Dropped 252 nodes (cum <= 0.55s)
Showing top 10 nodes out of 85
```

flat	flat%	sum%	cum	cum%	
-3.11s	2.81%	2.81%	-3.10s	2.80%	syscall.syscall
-2.77s	2.50%	5.31%	-2.76s	2.49%	runtime.pthread_cond_wait
-2.64s	2.38%	7.69%	-2.64s	2.38%	runtime.kevent
-1.17s	1.06%	8.75%	-1.17s	1.06%	runtime.madvise
0.87s	0.79%	7.96%	0.87s	0.79%	runtime.pthread_cond_timedwait_relative_np
-0.62s	0.56%	8.52%	-0.62s	0.56%	runtime.pthread_kill
0.21s	0.19%	8.33%	0.21s	0.19%	runtime.usleep
-0.10s	0.09%	8.42%	-0.10s	0.09%	runtime.typePointers.nextFast (inline)
0.06s	0.054%	8.37%	0.06s	0.054%	runtime.pthread_cond_signal
-0.06s	0.054%	8.42%	-0.08s	0.072%	runtime.typePointers.next

How PGO works ?

Inline Hot Function Calls

- Heap optimization:
- -2839m allocs, 1.66%

```
$ go tool pprof -sample_index=alloc_objects -diff_base heap.nopgo.pprof heap.withpgo.pprof
File: markdown.pgo
Type: alloc_objects
Time: Feb 16, 2025 at 10:54pm (CST)
Duration: 60s, Total samples = 170878373
Entering interactive mode (type "help" for commands, "o" for options)
(pprof) top
Showing nodes accounting for -2839721, 1.66% of 170878373 total
Dropped 22 nodes (cum <= 854391)
Showing top 10 nodes out of 94
```

flat	flat%	sum%	cum	cum%	
955765	0.56%	0.56%	955765	0.56%	gitlab.com/golang-commonmark/mdurl.(*URL).String
-737287	0.43%	0.13%	-1377226	0.81%	gitlab.com/golang-commonmark/markdown.ruleParagraph
-634885	0.37%	0.24%	-293550	0.17%	gitlab.com/golang-commonmark/markdown.(*StateInline)
-557075	0.33%	0.57%	-557075	0.33%	strings.genSplit
-548884	0.32%	0.89%	-730455	0.43%	gitlab.com/golang-commonmark/markdown.(*StateInline)
-462633	0.27%	1.16%	-462633	0.27%	bytes.growSlice
-450574	0.26%	1.43%	-439453	0.26%	gitlab.com/golang-commonmark/markdown.(*StateBlock).
425991	0.25%	1.18%	-159732	0.093%	gitlab.com/golang-commonmark/markdown.ruleNewline
-420527	0.25%	1.42%	-499721	0.29%	gitlab.com/golang-commonmark/markdown.ruleEmphasis
-409612	0.24%	1.66%	-223236	0.13%	gitlab.com/golang-commonmark/markdown.ruleHeading

```
(pprof) █
```


How PGO works ?

Inline Hot Function Calls

```
(pprof) peek gitlab.com/golang-commonmark/markdown.ruleParagraph
Showing nodes accounting for -1271586, 0.74% of 170878373 total
-----+-----
      flat  flat%   sum%        cum   cum%   calls calls% + context
-----+-----
              -1377226   100% | gitlab.com/golang-commonmark/markdown.ParserBlock.Tokenize
-737287  0.43%  0.43%   -1377226  0.81% | gitlab.com/golang-commonmark/markdown.ruleParagraph
              -657910  47.77% | gitlab.com/golang-commonmark/markdown.(*StateBlock).Lines
              24579   1.78% | gitlab.com/golang-commonmark/markdown.(*StateBlock).PushToken (inline)
              -6608   0.48% | gitlab.com/golang-commonmark/markdown.(*StateBlock).PushClosingToken (inline)
-----+-----
```

Where ruleParagraph called

```
+-----+
| gitlab.com/golang-commonmark/markdown.(*Markdown).Render
| gitlab.com/golang-commonmark/markdown.(*Markdown).Parse
| gitlab.com/golang-commonmark/markdown.ruleLinkify
| gitlab.com/golang-commonmark/markdown.ruleInline
| gitlab.com/golang-commonmark/markdown.ParserBlock.Parse
+-----+
```

w/o PGO

```
+-----+
| gitlab.com/golang-commonmark/markdown.(*Markdown).Render
| gitlab.com/golang-commonmark/markdown.(*Markdown).Parse
| gitlab.com/golang-commonmark/markdown.ruleLinkify
| gitlab.com/golang-commonmark/markdown.ruleInline
| gitlab.com/golang-commonmark/markdown.ParserBlock.Parse (inline)
|
+-----+
```

w/ PGO

How PGO works ?

Inline Hot Function Calls

- Observation:
 - Before, ParserBlock.Parse too big for inlining (Budget & Cost)
 - CPU Profile indicates it is hot, worth inlining (Breach Budget)
 - Overall performance optimized
- Why not do more inlining ?
 - Code Size: duplicated function body
 - Difficulty of traceback
 - CPU Instruction Cache: Long instructions -> cache invalidate
 - Compilation cost

How PGO works ?

Devirtualization of Interface Call

- What is Devirtualization of Interface Call ?

```

✓ type Processor interface { 3 usages 4 implementations new *
  — Process(x int) int 4 implementations
}

type FastProc struct{} 4 usages new *

✓ func (p FastProc) Process(x int) int { new *
  — return x * 2
}

```

```

✓ func callInterface(proc Processor, data []int) int { 2 usages new *
  — sum := 0
  — for i := range data {
    — — sum += proc.Process(data[i])
    — }
  — return sum
}

✓ func callDirect(proc FastProc, data []int) int { 2 usages new *
  — sum := 0
  — for i := range data {
    — — sum += proc.Process(data[i])
    — }
  — return sum
}

```


How PGO works ?

Devirtualization of Interface Call

- Benchmark tests

```

goos: darwin
goarch: arm64
pkg: pgo_example/devir
cpu: Apple M3 Pro
BenchmarkCalls/Interface-11      834381      1462 ns/op
BenchmarkCalls/Direct-11        1447210      812.2 ns/op
PASS

```

Disable Optimization

```

goos: darwin
goarch: arm64
pkg: pgo_example/devir
cpu: Apple M3 Pro
BenchmarkCalls/Interface-11    4038817    273.3 ns/op
BenchmarkCalls/Direct-11      4324443    280.9 ns/op
PASS

```

Enable Optimization

Devirtualization of Interface Call

Deep Dive

Check compiler decisions

```
$ go test -gcflags="-m" -bench=.
# pgo_example/devir [pgo_example/devir.test]
./devir.go:13:6: can inline FastProc.Process
./devir.go:17:6: can inline callInterface
./devir.go:25:6: can inline callDirect
./devir.go:28:22: inlining call to FastProc.Process
./devir.go:41:27: inlining call to callInterface
./devir.go:41:13: inlining call to fmt.Println
./devir.go:44:24: inlining call to callDirect
./devir.go:44:13: inlining call to fmt.Println
./devir.go:41:27: devirtualizing proc.Process to FastProc
./devir.go:41:27: inlining call to FastProc.Process
./devir.go:44:24: inlining call to FastProc.Process
./devir_test.go:15:17: inlining call to callInterface
./devir_test.go:15:17: devirtualizing proc.Process to FastProc
./devir_test.go:15:17: inlining call to FastProc.Process
./devir_test.go:22:14: inlining call to callDirect
./devir_test.go:22:14: inlining call to FastProc.Process
<autogenerated>:1: inlining call to FastProc.Process
```

Devirtualization of Interface Call

Deep Dive

```
func callInterface(proc Processor, data []int) int { 2 usages new *
    sum := 0
    for i := range data {
        sum += proc.Process(data[i])
    }
    return sum
}
```

Devirtualization result

```
func callInterface(proc FastProc, data []int) int { 2 usages new *
    sum := 0
    for i := range data {
        sum += proc.Process(data[i])
    }
    return sum
}
```


Devirtualization of Interface Call

Deep Dive

- Golang mechanism of interface call
- Implementation behind “iface” & “iTab”

```

type iface struct {
    tab *itab
    data unsafe.Pointer
}

```

```

// The first word of every non-empty interface type contains an *ITab.
// It records the underlying concrete type (Type), the interface type it
// is implementing (Inter), and some ancillary information.
//
// allocated in non-garbage-collected memory
type ITab struct {
    Inter *InterfaceType
    Type *Type
    Hash uint32 // copy of Type.Hash. Used for type switches.
    Fun [1]uintptr // variable sized. fun[0]==0 means Type does not implement Inter.
}

```


Devirtualization of Interface Call

Deep Dive

- Looking at the Assembly instructions, without devirtualization

```

r/devir.go:20)— MOVD—— R5, main.sum-16(SP)
r/devir.go:20)— MOVD—— 24(R0), R3   Load method address from interface table
r/devir.go:20)— MOVD—— (R2)(R4<<3), R5   Load argument
r/devir.go:20)— MOVD—— R1, R0   Setup receiver
r/devir.go:20)— MOVD—— R5, R1   Setup argument
r/devir.go:20)— PCDATA—— $1, $0   Register map for GC
r/devir.go:20)— CALL—— (R3)   Indirect call through interface table

```

Devirtualization of Interface Call

Deep Dive

- Looking at the ASM instructions, with devirtualization

<u>devir.go:28)</u> — <u>MOVD</u> —— (R0)(R2<<3), R1	<i>Load argument</i>
<u>devir.go:28)</u> — <u>MOVD</u> —— R1, R0	<i>Setup receiver</i>
<u>devir.go:28)</u> — <u>PCDATA</u> — \$1, \$0	<i>Register map for GC</i>
<u>devir.go:28)</u> — <u>CALL</u> —— main.FastProc.Process(SB)	<i>Direct call</i>

Devirtualization of Interface Call

Deep Dive

- Difference between function size

```
main.callInterface STEXT size=192 args=0x28 locals=0x28 funcid=0x0 align=0x0
```

```
main.callDirect STEXT size=160 args=0x18 locals=0x28 funcid=0x0 align=0x0
```


How PGO works ?

Devirtualization Hot Function Calls

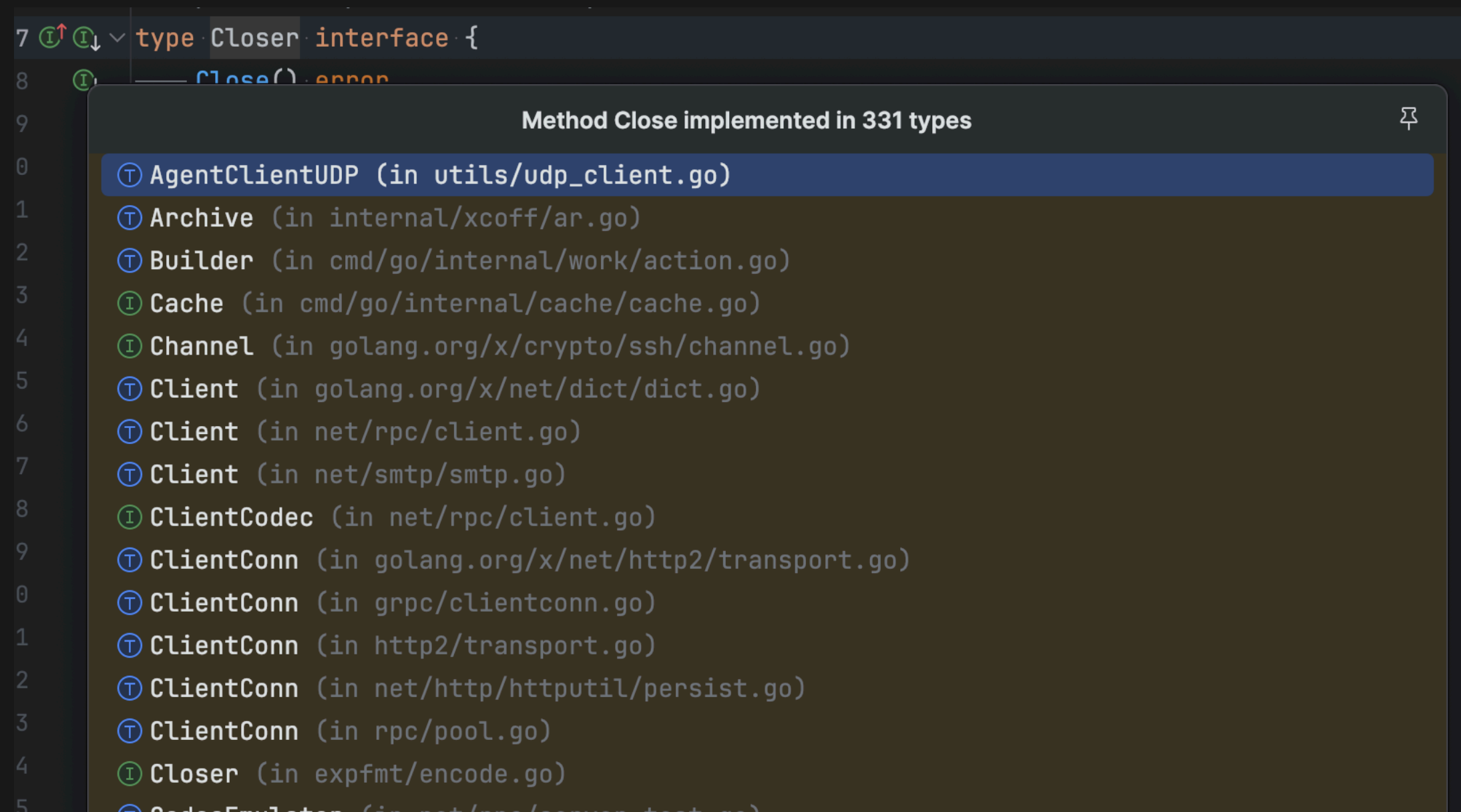
- Consideration as Function Inlining
- Duck typing: too many
 - `io.Closer.Close()`

```

— // Before devirtualization
— iproc.Process(x: 10)

— // After devirtualization
— if fp, ok := iproc.(FastProc); ok {
— |   fp.Process(x: 10)
— | } else {
— |   proc.Process(x: 10)
— | }

```



PGO

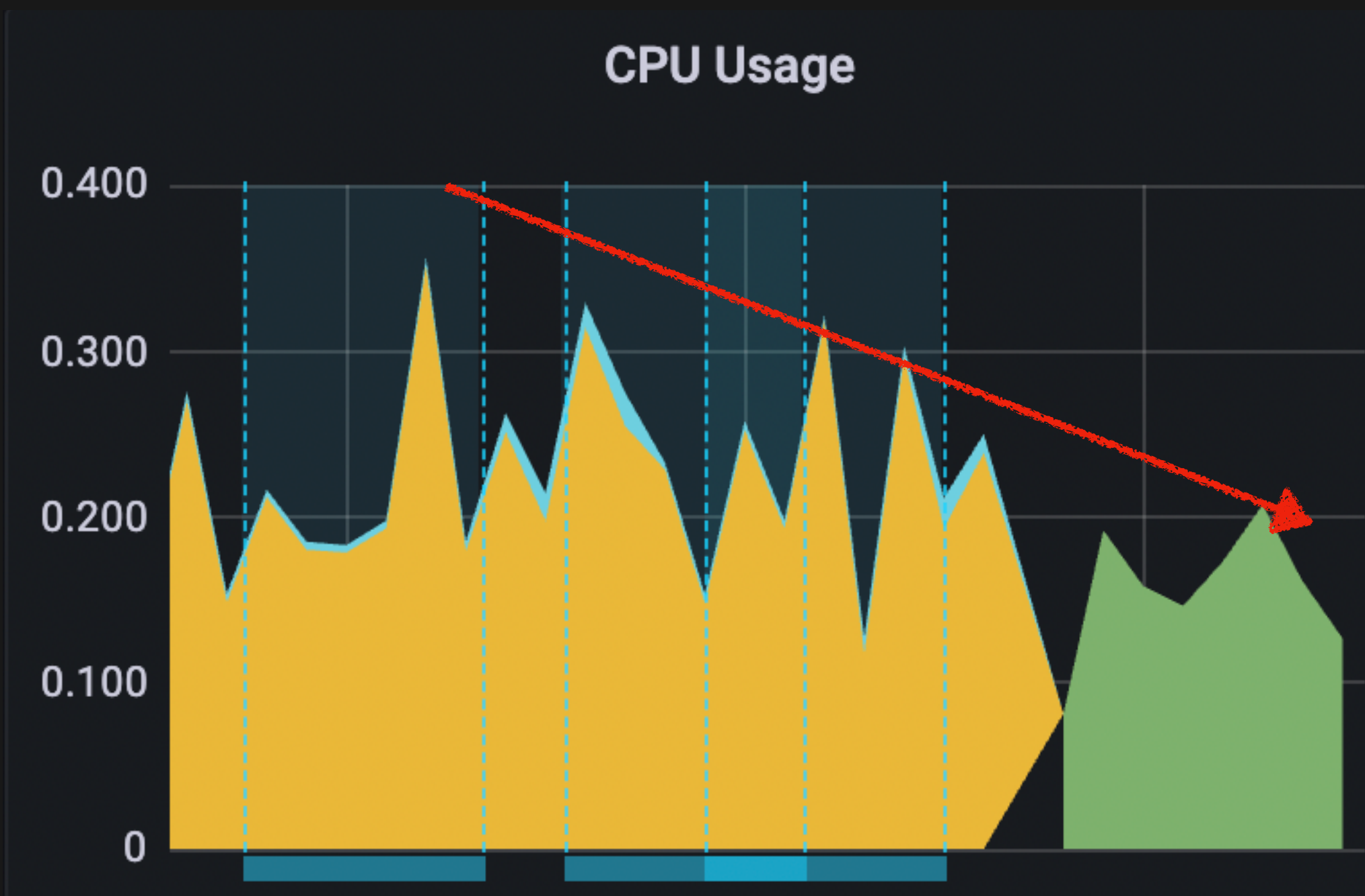
FAQ

- Will PGO optimize standard library ?
 - Yes
- Will PGO make app slower ?
 - Unlikely, inlining un-hot function won't affect anything a priori
- How does PGO affect build time and size ?
 - Slightly increased build time and size
- How should I handle different workload types ?
 - Different builds

Benefits of PGO In Real World

Benefits of PGO In Real World

titan/pagerduty



```
(pprof) top
Showing nodes accounting for -120ms, 12.00% of 1000ms total
Dropped 10 nodes (cum ≤ 5ms)
Showing top 10 nodes out of 601
flat flat% sum% cum cum%
-40ms 4.00% 4.00% -40ms 4.00% crypto/internal/bigmod.montgome
-40ms 4.00% 8.00% -60ms 6.00% runtime.scanobject
20ms 2.00% 6.00% 20ms 2.00% addMulVWx
20ms 2.00% 4.00% 20ms 2.00% crypto/internal/edwards25519/fi
20ms 2.00% 2.00% 20ms 2.00% encoding/json.appendCompact
-20ms 2.00% 4.00% -20ms 2.00% runtime.futex
-20ms 2.00% 6.00% -20ms 2.00% runtime.gopark
-20ms 2.00% 8.00% -20ms 2.00% runtime.lock2
-20ms 2.00% 10.00% -10ms 1.00% runtime.mallocgc
-20ms 2.00% 12.00% -30ms 3.00% runtime.unlock2
```


Benefits of PGO In Real World

titan/k-watch

```
(pprof) top
Showing nodes accounting for 160ms, 16.00% of 1000ms total
Dropped 8 nodes (cum ≤ 5ms)
Showing top 10 nodes out of 294
      flat flat% sum%      cum cum%
      70ms  7.00%  7.00%      70ms  7.00% runtime/internal/syscall.Syscall6
      -40ms  4.00%  3.00%      -40ms  4.00% runtime.memmove
      -40ms  4.00%  1.00%      -70ms  7.00% sigs.k8s.io/json/internal/golang/encoding/json.checkValid
      -30ms  3.00%  4.00%      -30ms  3.00% encoding/json.compact
      -30ms  3.00%  7.00%      -30ms  3.00% runtime.epollwait
      -30ms  3.00% 10.00%      -30ms  3.00% runtime.heapBitsSetType
      -30ms  3.00% 13.00%      10ms  1.00% runtime.schedule
      -30ms  3.00% 16.00%      -30ms  3.00% sigs.k8s.io/json/internal/golang/encoding/json.stateInString
      20ms  2.00% 14.00%      20ms  2.00% encoding/json.appendCompact
      -20ms  2.00% 16.00%      -30ms  3.00% encoding/json.stateEndValue
```


Benefits of PGO In Real World

titan/cmdb-agent

```
Showing nodes accounting for -120ms, 18.75% of 640ms total
Dropped 6 nodes (cum ≤ 3.20ms)
Showing top 10 nodes out of 265
..... flat flat% sum% ..... cum cum%
..... -30ms 4.69% 4.69% ..... -30ms 4.69% reflect.(*rtype).Kind
..... -30ms 4.69% 9.38% ..... -30ms 4.69% runtime.memmove
..... 30ms 4.69% 4.69% ..... 30ms 4.69% runtime.runqget
..... 30ms 4.69% 0% ..... 30ms 4.69% sync/atomic.(*Int32).Add
..... -20ms 3.12% 3.12% ..... -20ms 3.12% encoding/json.(*encodeState).string
..... -20ms 3.12% 6.25% ..... -30ms 4.69% encoding/json.compact
..... -20ms 3.12% 9.38% ..... -20ms 3.12% runtime.futex
..... -20ms 3.12% 12.50% ..... -20ms 3.12% runtime.nextFreeFast
..... -20ms 3.12% 15.62% ..... -20ms 3.12% runtime/internal/atomic.(*Int64).Add
..... -20ms 3.12% 18.75% ..... -20ms 3.12% runtime/internal/syscall.Syscall6
```

How to apply PGO ?

Easy

1. Compile binary:
 - `go build -o app .`
2. Run app with load, get profile:
 - `curl -o default.pgo "http://localhost:8080/debug/pprof/profile?seconds=30"`
3. Compile binary again (go build searches for default.pgo by default):
 - `go build -o app .`
 - `go build -pgo default.pgo app .`

Introducing AutoFDO

Auto Feedback Decision Optimization

- Google Research Paper
- App evolves, profile may be stale
 - New features, new hot functions
 - Library changes
- Periodically collect new profile data and merge the PGO file
 - `go tool pprof -proto default.pgo new.pprof > default.pgo`

Introducing AutoFDO

Auto Feedback Decision Optimization

Workflow:

1. Build and release an initial binary (without PGO).
2. Collect profiles from production.
3. When it's time to release an updated binary, build from the latest source and provide the production profile.
4. GOTO 2

Introducing AutoFDO

Gitlab CI Job

```
variables:
  PROJECT_NAME: titan/pagerduty
  REPO_NAME: go.planetmeican.com/titan/pagerduty
  TELEMETRY_ENDPOINT: http://pagerduty-telemetry.app-titan.svc.cluster.local:18888
include:
  - project: 'meican-cd/ci-template'
    ref: main
    file: jobs/pgo.yml
stages:
  - pgo
```

Introducing AutoFDO

Gitlab CI Job

pgo

 pgo:meican2



 pgo:sandbox



```
Connecting to pagerduty-telemetry.app-titan.svc.cluster.local:18888 (10.100.37.116:18888)
saving to 'pgo/cpu-sandbox-20250217085307.pprof'
cpu-sandbox-20250217 100% |*****| 9850 0:00:00 ETA
'pgo/cpu-sandbox-20250217085307.pprof' saved
[detached HEAD bb0ffe9] perf: auto download sandbox PGO at 20250217085307. please build after this commit.
2 files changed, 0 insertions(+), 0 deletions(-)
mode change 100644 => 120000 default.pgo
create mode 100644 pgo/cpu-sandbox-20250217085307.pprof
remote:
remote: View merge request for perf-pgo:
remote: https://gitlab.planetmeican.com/titan/pagerduty/-/merge\_requests/120
remote:
To https://gitlab.planetmeican.com/titan/pagerduty.git
30ce845..bb0ffe9 HEAD -> perf-pgo
Job succeeded
```

Overview 0

Commits 2

Pipelines 4

Changes 4

Feb 17, 2025



perf: auto download sandbox PGO at 20250217085307. please build after this commit.
marvin_bot authored 5 minutes ago



perf: pgo
Huang yanshuo authored 1 hour ago

PGO

Takeaways

- Maximum gain:
 - Run on production / during busy time
- AutoFDO: do PGO periodically - Gitlab CI
- -10% CPU cost
 - EC2: 850+ CPU Cores
- ROI

References

- <https://go.dev/blog/pgo>
- <https://research.google/pubs/autofdo-automatic-feedback-directed-optimization-for-warehouse-scale-applications/>
- <https://pkg.go.dev/golang.org/x/tools/internal/refactor/inline>
- <https://planetmeican.atlassian.net/wiki/spaces/DEV/pages/661717002/Golang+Profile-guided+optimization+PGO#AutoFDO>

Fin.
Question ?